

Object Oriented Programming Review

What is Object-Oriented Programming?

Object-oriented programming: A programming style in which developers treat everything in their code like a real-world object. It is popularly called OOP. The four key principles that help you organize and manage code effectively are encapsulation, inheritance, polymorphism, and abstraction

Classes: The blueprint for creating objects. Every single object created from a class has attributes that define data and methods that determine the behaviors of the objects.

What is Encapsulation?

Encapsulation: The bundling of the attributes and methods of an object into a single unit. It lets you hide the internal state of the object behind a simple set of public methods and attributes that act like doors. Behind those doors are private attributes and methods that control how the data changes and who can see it.

Example of Encapsulation: If you want to track a wallet balance, you will allow deposit and withdrawal, but you won't want anyone to tamper with the wallet balance itself:

```
class Wallet:
    def __init__(self, balance):
        self.__balance = balance # Private attribute

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount # Add to the balance safely

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount # Remove from the balance safely
```

```
account = Wallet(500)

print(account.__balance) # AttributeError: 'Wallet' object
has no attribute '__balance'
```

Difference Between Prefixing Attributes with Single and Double Underscore:

Prefixing attributes and methods with a single underscore means they are meant for internal use. This is a convention, and it doesn't enforce accessing attributes from the outside. Prefixing attributes and methods with a double underscore effectively prevents them from being accessed from outside of their class.

What Are Getters and Setters?

Getters and Setters: Methods that let you control how the attributes of a class are accessed and modified. You retrieve values with getters and you set values with setters.

Properties: They connect getters and setters, and allow access to data. They run extra logic behind the scenes when you get, set, or delete values.

Why Properties Instead of Methods: Properties are used instead of methods for better readability and cleaner code. They let you access values with dot notation, like regular attributes, without parentheses.

Creating a Getter: To create a getter, you use the `@property` decorator. Here's a getter that gets the radius of a circle:

```
class Circle:

    def __init__(self, radius):

        self._radius = radius

    @property

    def radius(self): # A getter to get the radius

        return self._radius

    @property

    def area(self): # A getter to calculate area
```

```
return 3.14 * (self._radius ** 2)
```

```
my_circle = Circle(3)
```

```
print(my_circle.radius) # 3
```

```
print(my_circle.area) # 28.26
```

Creating a Setter: To create the setter that will set the radius, you have to define another method with the same name and use `@<property_name>.setter` above it:

```
class Circle:
```

```
    def __init__(self, radius):
```

```
        self._radius = radius
```

```
    @property
```

```
    def radius(self): # A getter to get the radius
```

```
        return self._radius
```

```
    @radius.setter
```

```
    def radius(self, value): # A setter to set the radius
```

```
        if value <= 0:
```

```
            raise ValueError('Radius must be positive')
```

```
        self._radius = value
```

```
my_circle = Circle(3)
```

```
print('Initial radius:', my_circle.radius) # Initial radius:
3
```

```
my_circle.radius = 8
```

```
print('After modifying the radius:', my_circle.radius) #  
After modifying the radius: 8
```

How Python Handles Getters and Setters: Once you define getters and setters, Python automatically calls them under the hood whenever you use normal attribute syntax this way:

```
my_circle.radius # This will call the getter  
my_circle.radius = 4 # This will call the setter
```

When setting a value, you should not assign to the property name itself because that will cause a `RecursionError`. Use a separate internal name, often with an underscore, to store the value.

Deleter: After setting and getting a value with setter and getter, you can control how it is deleted with a `deleter`. A deleter runs custom logic when you use the `del` statement on a property. To create a deleter, you use the `@<property name>.deleter` decorator.

```
# Deleter  
  
@radius.deleter  
def radius(self):  
    print("Deleting radius...")  
    del self._radius
```

What Is Inheritance and How Does It Promote Code Reuse?

Inheritance: The process by which a child class uses the attributes and methods of a parent class. Inheritance promotes code reuse, provides clear hierarchies, and customizes behavior without rewriting everything. To implement inheritance, a child class takes in the name of a parent class:

```
class Parent:  
    # Parent attributes and methods  
  
class Child(Parent):  
    # Child inherits, extends, and/or overrides where  
    necessary
```

Single and Multiple Inheritance: When a child class inherits properties and methods from a single parent, as you can see above, the process is called single inheritance. When a child class inherits properties and methods from more than one parent, that is multiple inheritance. Here's the syntax for that:

```
class Parent:
    # Attributes and methods for Parent

class Child:
    # Attributes and methods for Child

class GrandChild(Parent, Child):
    # GrandChild inherits from both Parent and Child
    # GrandChild can combine or override behavior from each
```

`super()` Function: A function that lets you call a method from a parent class, when a class has a different implementation of that method, or it extends the method, without duplicating code.

What Is Polymorphism and How Does It Promote Code Reuse?

Polymorphism: The OOP principle that lets different classes use the same method name, but each class implements it differently when called. Here's the syntax for it:

```
class A:
    def action(self): ...

class B:
    def action(self): ...

class C:
    def action(self): ...
```

```
Class().method() # Works for A, B, or C
```

Inheritance-based polymorphism: A parent sets up a method, and each child class twists it to their use.

What is Name Mangling and How Does it Work?

Name Mangling: A process in which Python internally renames an attribute prefixed with a double underscore by adding an underscore and the class name as a prefix, turning `attribute` into `ClassName attribute`.

The Purpose of Name Mangling: The main purpose of name mangling is to prevent accidental attribute and method overriding when you use inheritance. Here's a code that makes that more understandable:

```
class Parent:
    def __init__(self):
        self.__data = 'Parent data'

class Child(Parent):
    def __init__(self):
        super().__init__()
        self.__data = 'Child data'

c = Child()
print(c.__dict__) # {'_Parent__data': 'Parent data',
'_Child__data': 'Child data'}
```

What Is Abstraction and How Does It Help Keep Complex Systems Organized?

Abstraction: A programming concept in which complex implementation details of object or system are hidden and only the essential features are shown. In Python and other programming languages, abstraction simplifies complex systems by increasing reusability.

Example of Abstraction: A good example of abstraction in everyday life is a car letting you just use the wheel, pedals, and shifter without knowing how the engine or brakes work.

How Python Implements Abstraction: Python implements abstraction through the `abc` module. The module provides the `ABC` class (abstract base class) and the `@abstractmethod` decorator. An abstract base class (ABC) defines the common methods and properties subclasses must implement. It can't be instantiated.

How Abstract Method is Defined: An abstract method is defined with `@abstractmethod` and must be overridden in subclasses, even if it has a default implementation. The basic syntax of abstraction looks like this:

```
from abc import ABC, abstractmethod

# Define an abstract base class
class AbstractClass(ABC):
    @abstractmethod
    def abstract_method(self):
        pass

# Concrete subclass that implements the abstract method
class ConcreteClassOne(AbstractClass):
    def abstract_method(self):
        print('Implementation in ConcreteClassOne')

# Another concrete subclass
class ConcreteClassTwo(AbstractClass):
    def abstract_method(self):
        print('Implementation in ConcreteClassTwo')
```